

MACHINE LEARNING LAB MANUAL



Department of Computer Science Engineering

Amrita School of Engineering

Amrita Vishwa Vidyapeetham, Amaravati Campus

Name: SreeDurga

Roll: CSEA24020

1) Create a NumPy array containing the integers from 1 to 20. Perform the following operations:

1. Display the array.
2. Find the shape of the array.
3. Find the maximum and minimum values.
4. Calculate the mean and standard deviation.
5. Reshape the array into a 4×5 matrix.
6. Display the second row of the matrix.
7. Display the third column of the matrix.

Code:

```
import numpy as np

arr=np.arange(1,21)

print(arr)

print(arr.shape)

print(arr.max())

print(arr.min())

print(arr.mean())

print(arr.std())

reshaped=arr.reshape(4,5) # reshapes in a 4*5 matrix

print(reshaped)

print(reshaped[1]) # prints 2nd row

print(reshaped[:,2])
```

Output:

```
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]
(20,)
20
1
10.5
5.766281297335398
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]
 [16 17 18 19 20]]
[ 6  7  8  9 10]
[ 3  8 13 18]
```

2) Create two NumPy arrays:

A = [10, 20, 30, 40, 50]

B = [5, 10, 15, 20, 25]

Perform:

1. Addition of A and B.
2. Subtraction of B from A.
3. Element-wise multiplication.
4. Element-wise division.
5. Find the sum of all elements in A.

Code:

```
import numpy as np

a=np.array([10,20,30,40,50])

b=np.array([5,10,15,20,25])

#addition

add=a+b

print(add)

#subtaction

subtract=a-b

print(subtract)
```

```
#element wise multiplication
```

```
mul=a*b
```

```
print(mul)
```

```
# element wise division
```

```
div=a/b
```

```
print(div)
```

```
#sum of elements in an array
```

```
arraysum=np.sum(a)
```

```
print(arraysum)
```

Output:

```
[15 30 45 60 75]
[ 5 10 15 20 25]
[ 50 200 450 800 1250]
[2. 2. 2. 2. 2.]
150
```

3) Create a DataFrame using the following student data:

Roll No	Name	Marks
101	Ravi	85
102	Priya	92
103	Kiran	78
104	Anu	88
105	Sita	95

Perform the following operations:

1. Display the DataFrame.
2. Display the first three records.

3. Find the average marks.
4. Find the student with the highest marks.
5. Find the student with the lowest marks.
6. Add a new column called "Grade" using:
 - o A for Marks ≥ 90
 - o B for Marks between 80 and 89
 - o C for Marks < 80

Code:

```
import pandas as pd

data={

    "Roll No":[101,102,103,104,105],

    "Name":["Ravi","Priya","Kiran","Anu","Sita"],

    "Marks":[85,92,78,88,95]

}

df=pd.DataFrame(data)

print(df)

print(df.head(3))

print(df["Marks"].mean())

print(df["Marks"].max())

print(df["Marks"].min())

# 2. Define the grading logic

def assign_grade(mark):

    if mark >= 90: return 'A'

    elif mark >= 80: return 'B'

    elif mark >= 70: return 'C'
```

```
elif mark >= 60: return 'D'
```

```
else: return 'F'
```

```
# 3. Add the 'Grade' column using .apply()
```

```
df['Grade'] = df['Marks'].apply(assign_grade)
```

```
print(df)
```

Output:

```
Roll No    Name  Marks
0      101    Ravi    85
1      102   Kiran    57
2      103   Priya    90
3      104 Preethi    67
Roll No    Name  Marks
0      101    Ravi    85
1      102   Kiran    57
2      103   Priya    90
74.75
90
57
Roll No    Name  Marks  Grade
0      101    Ravi    85     B
1      102   Kiran    57     F
2      103   Priya    90     A
3      104 Preethi    67     D
```

4) Create a DataFrame containing the following information:

Product	Price	Quantity
Laptop	50000	10
Mouse	500	50
Keyboard	1200	30
Monitor	10000	25

Perform:

1. Calculate the total value of each product using: Total Value = Price × Quantity
2. Add the Total Value column.
3. Find the product with the highest total value.
4. Calculate the overall inventory value.

Code:

```
import pandas as pd

data={

    "Product":["Laptop","Mouse","Keyboard","Monitor"],

    "Price":[50000,500,1200,10000],

    "Quantity":[10,50,30,25]}

df=pd.DataFrame(data)

print(df)

df['Total Value']=df['Price']*df['Quantity']

print(df)

print(df['Total Value'].max())

print(df['Total Value'].sum())
```

Output:

	Product	Price	Quantity	
0	Laptop	50000	10	
1	Mouse	500	50	
2	Keyboard	1200	30	
3	Monitor	10000	25	
	Product	Price	Quantity	Total Value
0	Laptop	50000	10	500000
1	Mouse	500	50	25000
2	Keyboard	1200	30	36000
3	Monitor	10000	25	250000
				500000
				811000

5) Download the Iris dataset from Scikit-learn and perform the following:

1. Load the dataset into a Pandas DataFrame.
2. Display the first five records.
3. Find the number of rows and columns.
4. Display column names.
5. Check for missing values.
6. Generate summary statistics using describe().
7. Find the number of samples belonging to each class.

Code:

```
from sklearn.datasets import load_iris

import pandas as pd

# Load the iris dataset

iris = load_iris()

# Create a DataFrame with the feature data and feature names

df = pd.DataFrame(data=iris.data, columns=iris.feature_names)

df['target'] = iris.target

# Display the first 5 records

print(df.head())

print(df.shape)

print(iris.feature_names)

print(df.describe())

print(df.isnull().sum())
```

Output:

```

    sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                    5.1                3.5                1.4                0.2
1                    4.9                3.0                1.4                0.2
2                    4.7                3.2                1.3                0.2
3                    4.6                3.1                1.5                0.2
4                    5.0                3.6                1.4                0.2

    target
0        0
1        0
2        0
3        0
4        0
(150, 5)
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
    sepal length (cm)  sepal width (cm)  petal length (cm)  \
count          150.000000          150.000000          150.000000
mean           5.843333           3.057333           3.758000
std            0.828066           0.435866           1.765298
min            4.300000           2.000000           1.000000
25%            5.100000           2.800000           1.600000
50%            5.800000           3.000000           4.350000
75%            6.400000           3.300000           5.100000
max            7.900000           4.400000           6.900000
...
petal length (cm)    0
petal width (cm)    0
target              0
dtype: int64
```